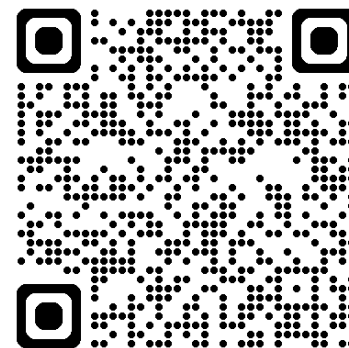




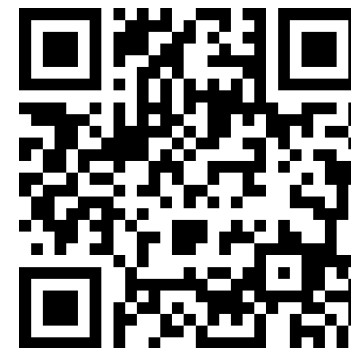
深度學習 Deep Learning

Recurrent Neural Networks (RNN)

Instructor: 林英嘉 (Ying-Jia Lin)
2025/10/22



[Course GitHub](#)



[Slido # DL1022](#)

Outline

- Introduction to Natural Language Processing (NLP) [30 min]
- Recurrent Neural Networks (RNN) [50 min]
- NLP code [50 min]
- Quiz [20 min]



The Revolution of ChatGPT

ChatGPT came out in November, 2022.

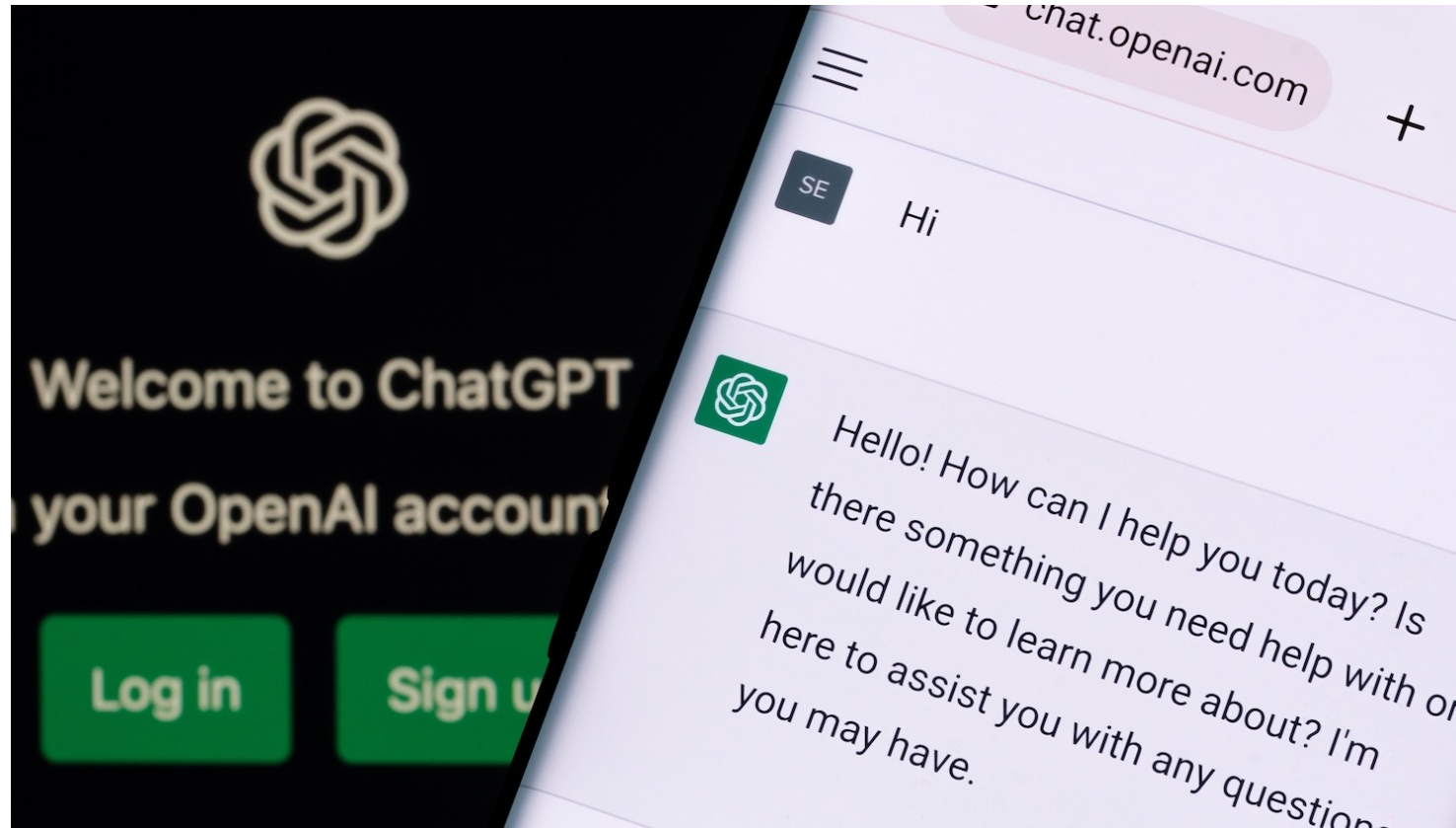


Figure source: <https://poole.ncsu.edu/thought-leadership/article/lets-chat-about-chatgpt/>

大型語言模型的應用

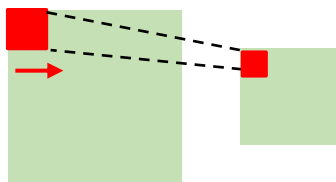


翻譯、聊天機器人、寫故事、
摘要、教學、
寫程式、寫作業 ...

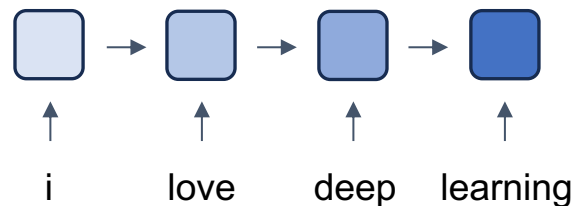


深度學習常見模型架構

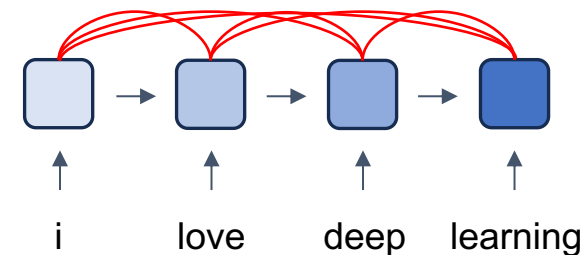
CNN (著重局部資訊)



RNN (著重記憶力)



Transformer (著重全局資訊與記憶力)



Natural Language Processing (NLP)

電腦是如何學習自然語言呢？

NLP 三大秘辛

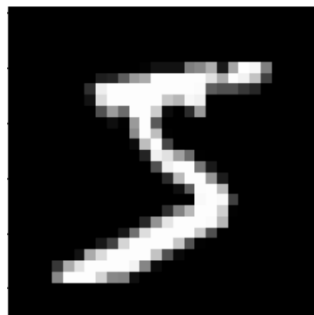
1 每個字，都是一段向量

2 每個 NLP 模型都有一個字典 

3 自然語言處理模型需要記憶力 



影像 vs. 文字



→ 28 x 28 的矩陣

Apple

→ ?

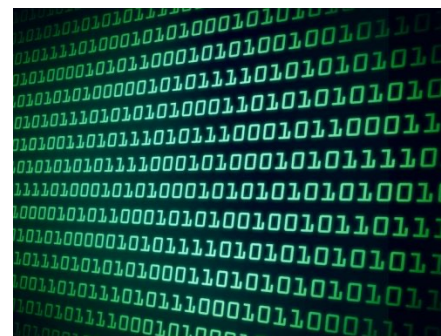


Figure source: <https://kopu.chat/wp-content/uploads/2017/04/binary-507x380.jpeg>



每個字，都是一段向量

可以自行設定向量長度

she = [0.110960 0.032115 -0.004809]

her = [0.110950 0.032117 -0.004909]

grandpa = [0.023115 0.708091, 0.512345]

grandma = [0.023113 0.708101, 0.512445]

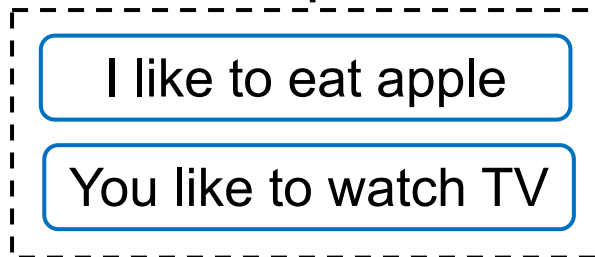


Word Representations



Basic Approach

Corpus



→ Vocabulary

apple	[1, 0, 0, 0, 0, 0, 0, 0]
eat	[0, 1, 0, 0, 0, 0, 0, 0]
I	[0, 0, 1, 0, 0, 0, 0, 0]
like	[0, 0, 0, 1, 0, 0, 0, 0]
watch	[0, 0, 0, 0, 1, 0, 0, 0]
to	[0, 0, 0, 0, 0, 1, 0, 0]
⋮	

One hot encodings with each vector size equal to the vocab size (8 in this case)

Pros: handy, training-free
Cons: sparse matrix which occupies much memory



nn.Embeddings

```
1 word_to_idx = {"好吃": 0, "棒": 1, "给力": 2}
2 embeds = torch.nn.Embedding(3, 5) # 2 words in vocab, 5 dimensional embeddings
```

```
1 lookup_tensor = torch.tensor(
2     [
3         word_to_idx["好吃"],
4         word_to_idx["棒"],
5         word_to_idx["给力"],
6     ],
7     dtype=torch.long,
8 )
9 word_embed = embeds(lookup_tensor)
10 print(word_embed)
```

[7]

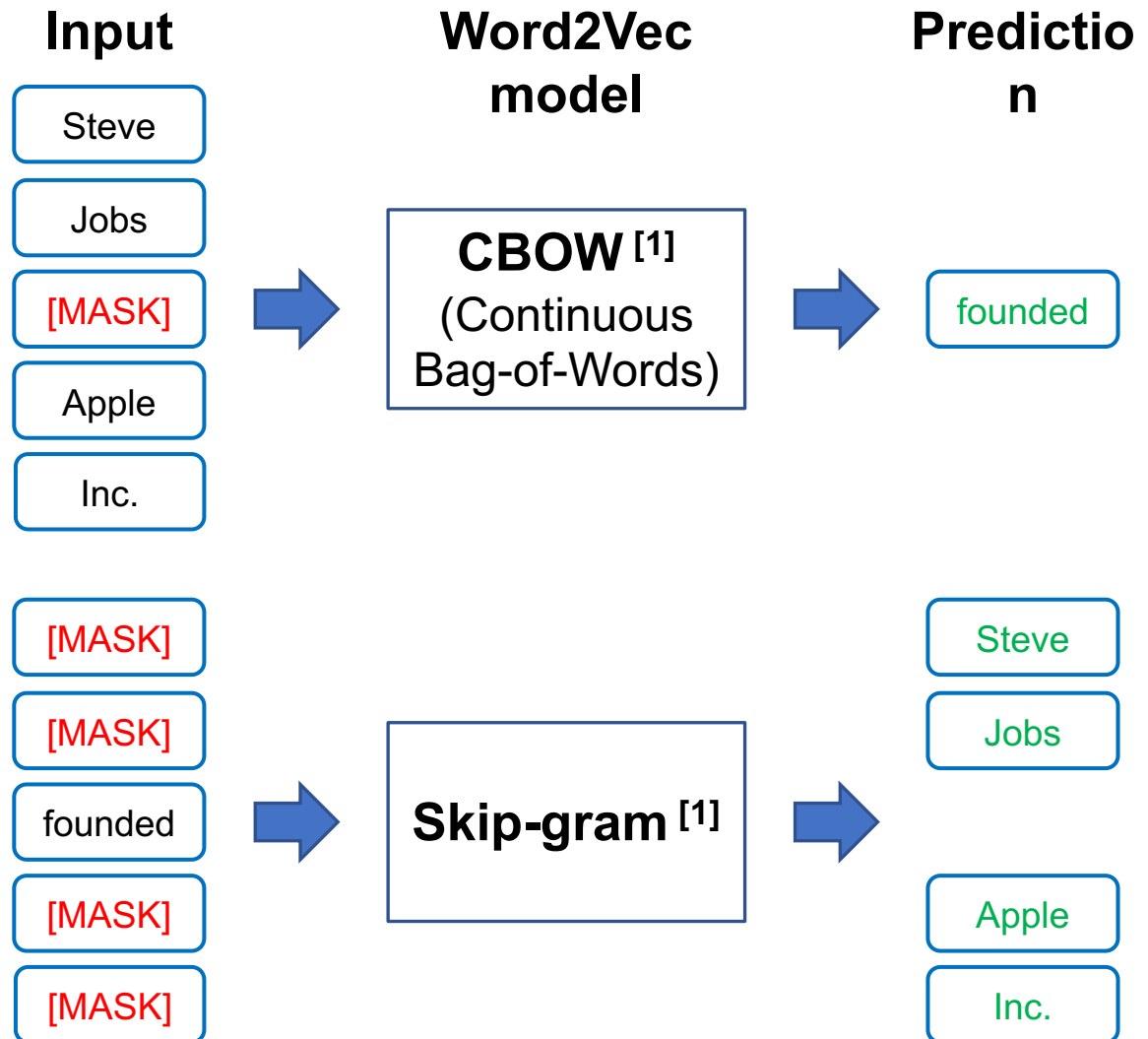
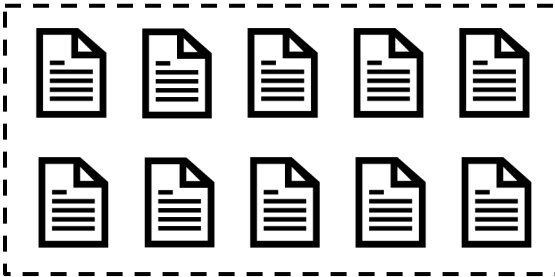
```
... tensor([[ 2.2014,  1.9880,  1.5726, -0.1552,  0.0475],
          [-0.0540, -1.5926, -0.5808, -0.6631, -0.3321],
          [-0.3562,  1.0019,  0.8178, -0.9174, -0.0707]],
        grad_fn=<EmbeddingBackward0>)
```



Word Embeddings (Word2Vec)

[1] Mikolov et al. "Distributed representations of words and phrases and their compositionality." NeurIPS 2013.

Large Corpora
(E.g., Wikipedia)



After Training Word Embeddings

- The outputs are dense vectors with a fixed dimension size:

		Dimension size (e.g., 300)					
Vocabulary size (e.g., 30,000)	apple	-0.110960	0.016115	-0.004809	0.033589	0.121455	...
	banana	-0.027713	-0.015676	0.003314	0.077602	0.159718	...
	...						
	...						

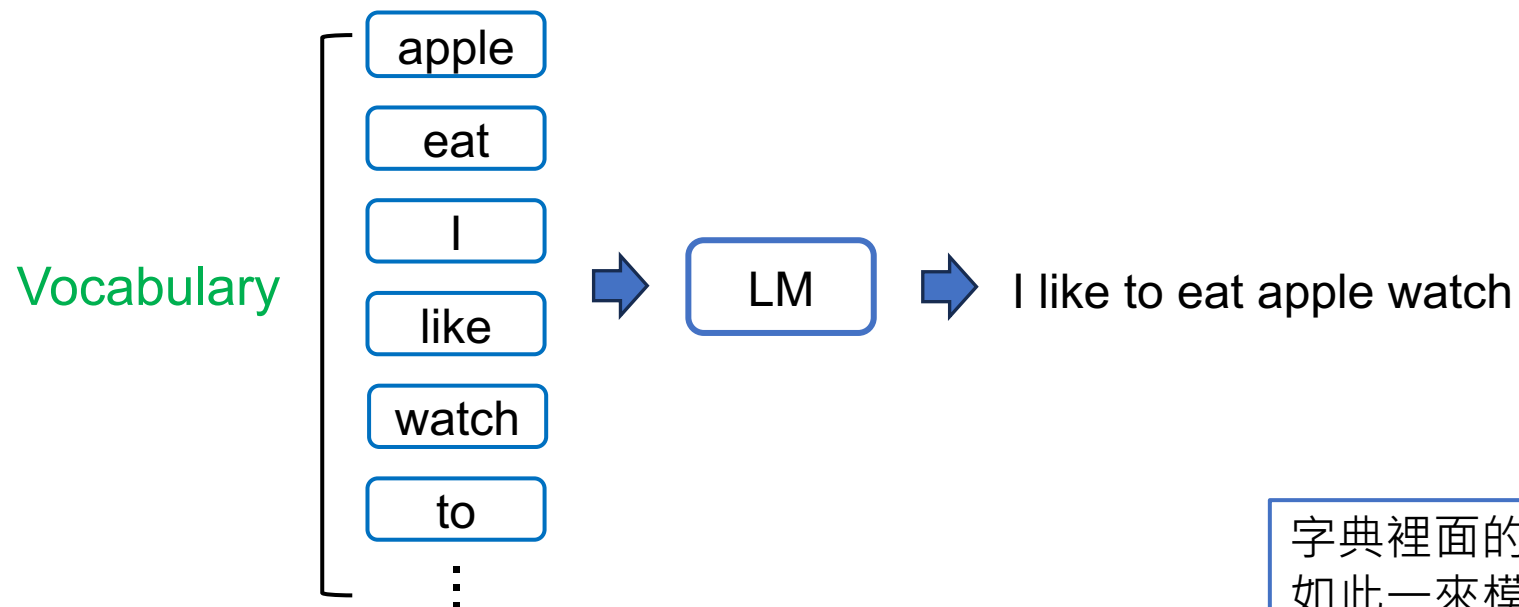


NLP 三大秘辛

- 1 每個字，都是一段向量
- 2 每個 NLP 模型都有一個字典 
- 3 自然語言處理模型需要記憶力 



每個 NLP 模型都有一個字典



字典裡面的內容是語言模型能理解的最小單元，
如此一來模型才能夠進行目標任務 (e.g., 生成)



... (你平常操作 ChatGPT 的結果)



字典實作

字典的變數
(key: 字, value: index)

通常我們會預先準備 <pad> 跟 <unk>
前者代表 padding，後者代表 unknown words

```
1 vocab = {'<pad>':0, '<unk>':1}
2
3 # 使字詞不重複
4 words = sorted(set(words))
5 for idx, word in enumerate(words):
6     # 一開始已經放兩個進去 dictionary 了
7     idx = idx + 2
8     # 將 word to id 放到 dictionary
9     vocab[word] = idx
```

Python

字典在NLP中的實作方式就真的採用Python dictionary



[注意事項] 字典實作

字典是經由訓練資料集的字所建立的

```
word_to_idx = {"<pad>": 0, "<unk>": 1, "好吃": 2, "棒": 3, "给力": 4}
embeds = torch.nn.Embedding(5, 5) # 5 words in vocab, 5 dimensional embeddings
```

```
def get_word_id(word, vocab, unk_idx: int = 1):
    return vocab.get(word, unk_idx) ← 如果某字不存在於字典中，則通常給予 <unk> 的 id
```

```
lookup_tensor = torch.tensor([
    get_word_id("好吃", word_to_idx),
    get_word_id("不棒", word_to_idx),
    get_word_id("<unk>", word_to_idx),
],)
word_embed = embeds(lookup_tensor)
print(word_embed)

tensor([[ -0.5574,  0.1768,  0.9588,  1.2837, -1.7256],
        [-3.4725, -1.0435, -0.0538, -0.0625, -0.4173],
        [-3.4725, -1.0435, -0.0538, -0.0625, -0.4173]]
grad_fn=<EmbeddingBackward0>)
```



NLP 三大秘辛

- 1 每個字，都是一段向量
- 2 每個 NLP 模型都有一個字典 
- 3 自然語言處理模型需要記憶力 



語言的模式

★ 上下文非常重要

句子1：

小明昨天遇到小美，他對她說：「下次一起去看電影吧！」
，請問小明下次想去跟誰看電影？

句子2：

「庭院深深深幾許」，請問有幾個深？

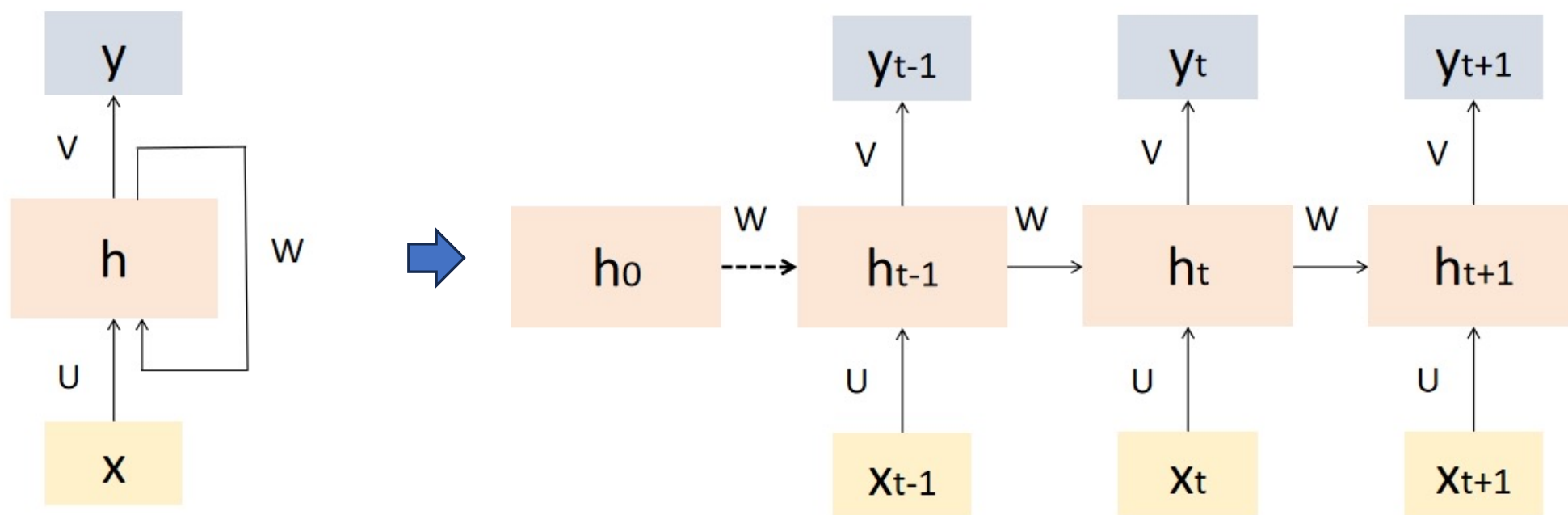


Recurrent Neural Networks

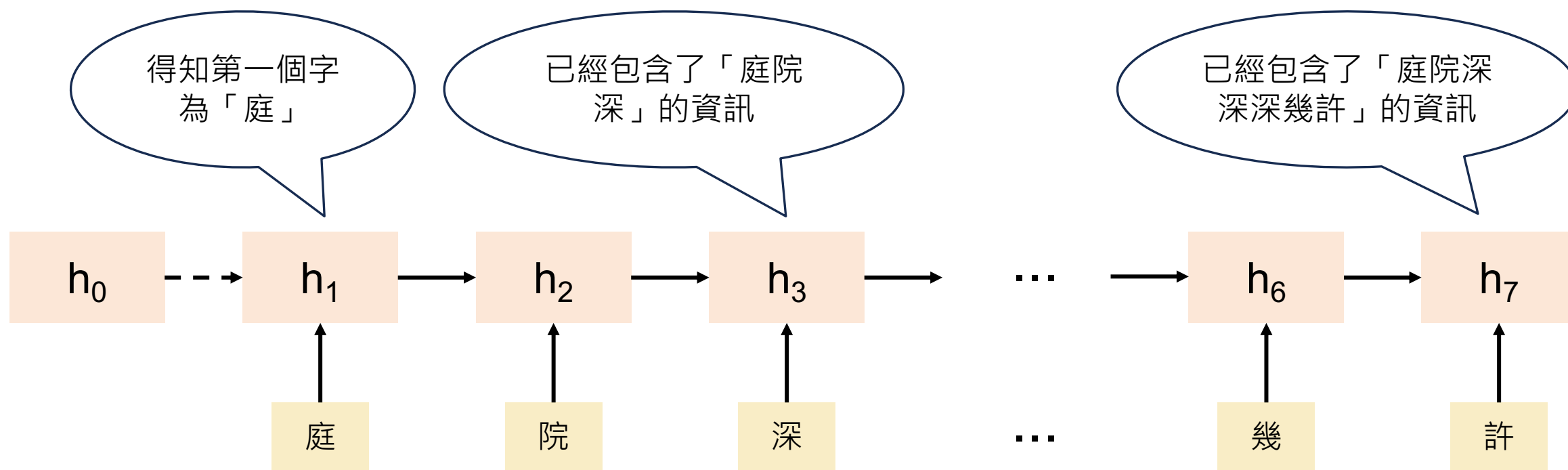
RNN

Recurrent Neural Networks (RNN)

- Recurrent Neural Networks (RNNs) are a type of artificial neural network designed to handle sequential data by **capturing temporal dependencies**.
 - RNN 的“遞迴”是指它在每一個時間步中重複使用相同的參數（同一組權重），並把先前的隱藏狀態傳到下一個時間步



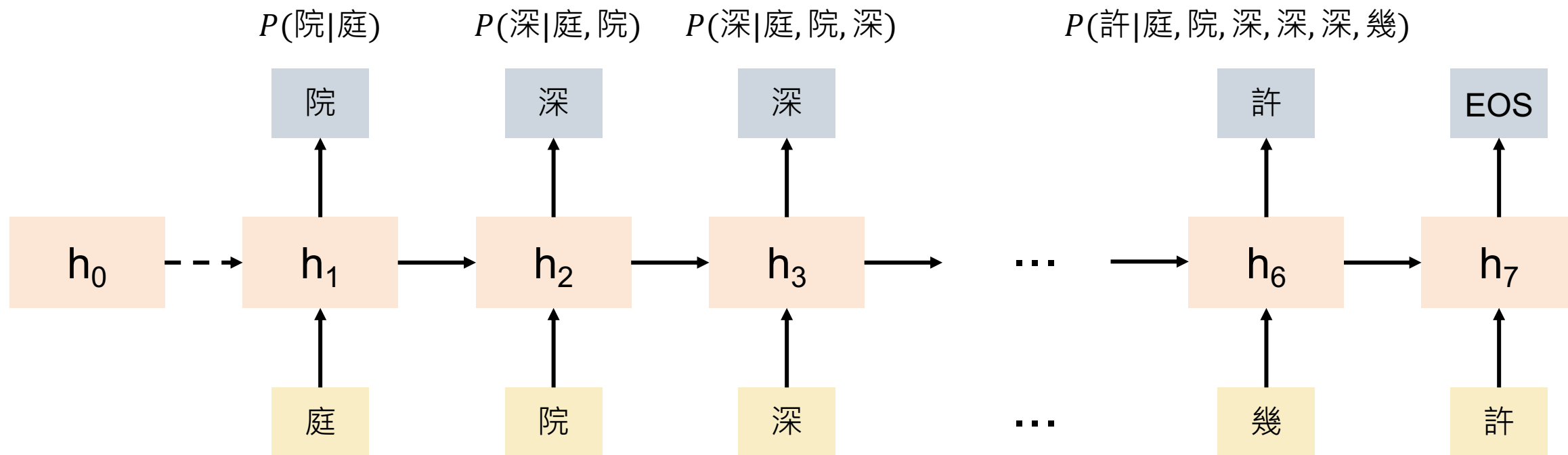
RNN 的記憶力意義



RNN 以文字接龍進行訓練

EOS: End of Sentence

$P(\text{EOS}|\text{庭, 院, 深, 深, 深, 幾, 許})$



目標函數： $\log(P(\text{院}|\text{庭}) \times P(\text{深}|\text{庭, 院}) \times P(\text{深}|\text{庭, 院, 深}) \times \dots \times P(\text{EOS}|\text{庭, 院, 深, 深, 深, 幾, 許}))$



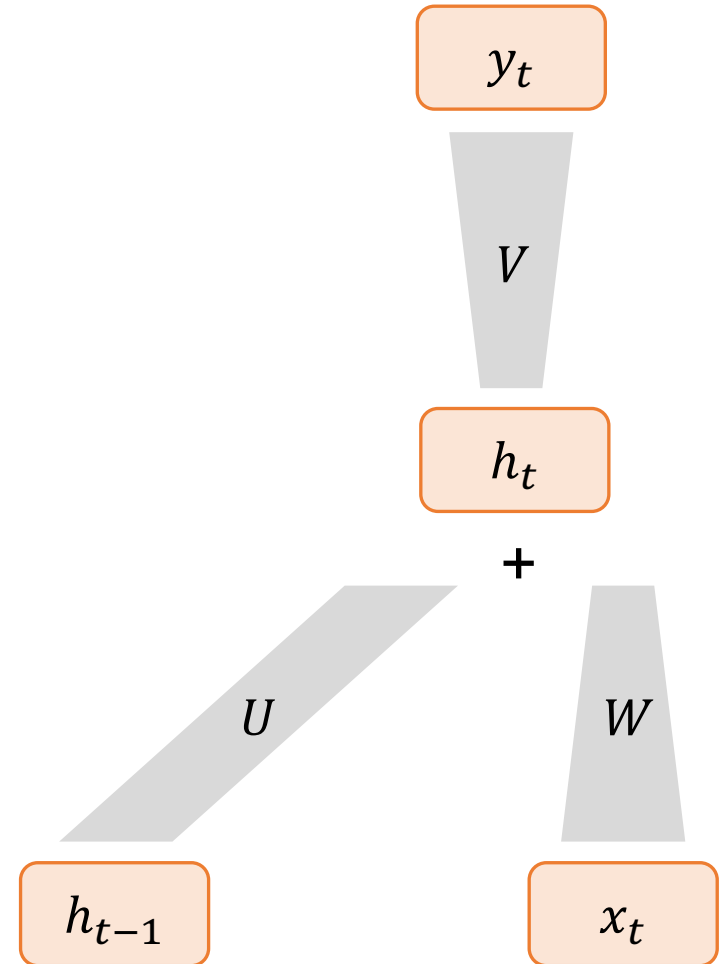
RNN (本頁省略 bias)

➤ The equation on step t is:

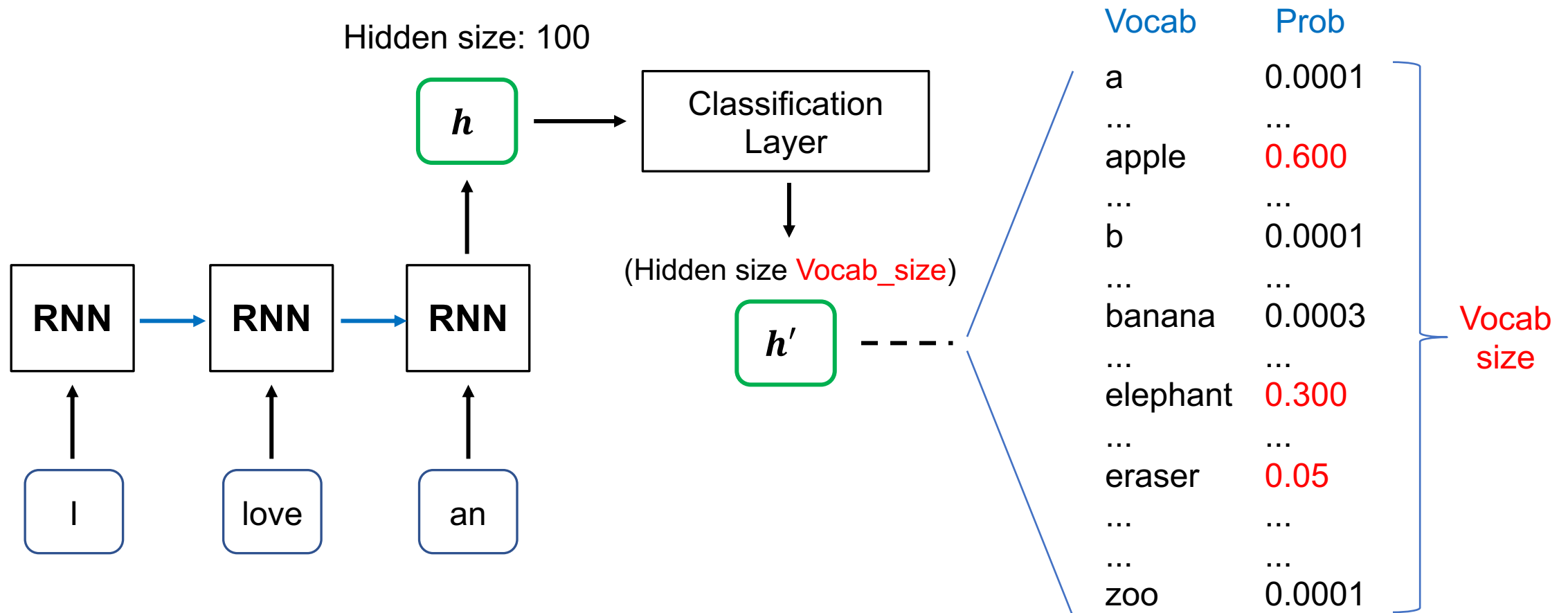
$$h_t = \tanh(Wx_t + Uh_{t-1})$$

$$y_t = Vh_t$$

, where $\tanh$ is an activation function.



Text Generation with RNN



為什麼我們通常不用 MLP 來做 NLP ?

➤ Lack of Sequence Modeling:

- In NLP tasks, understanding the sequence of words and their dependencies is crucial for accurate predictions.

➤ Fixed Input Size:

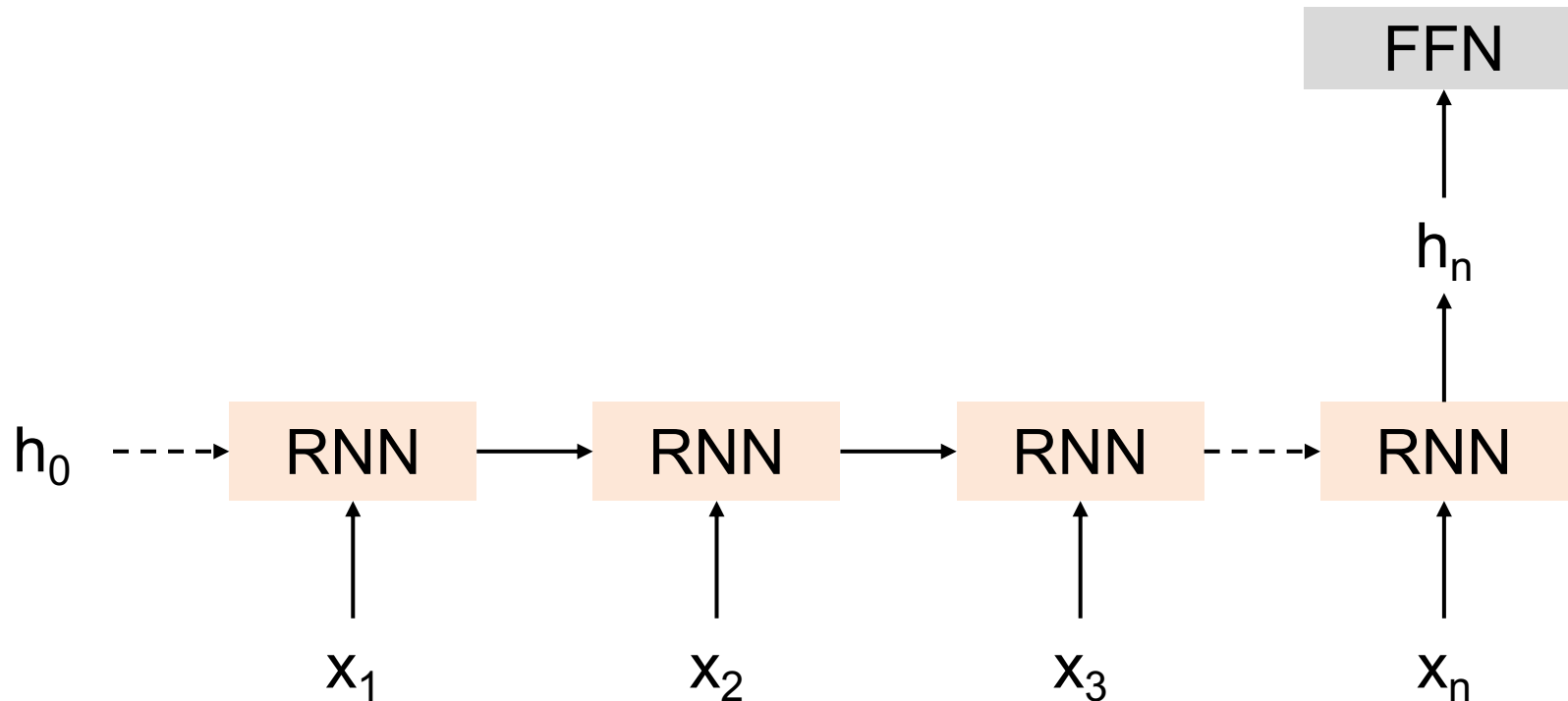
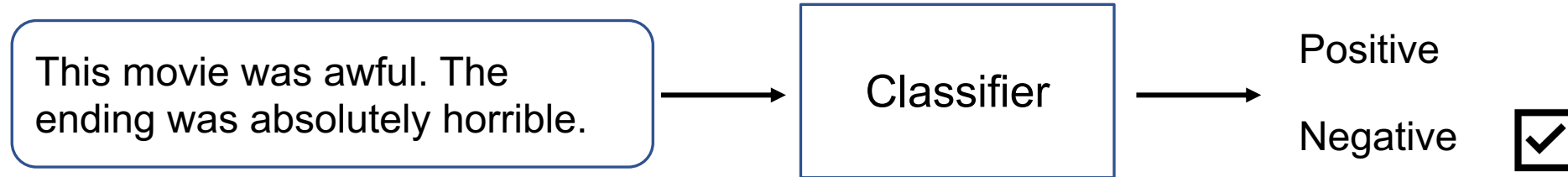
- FFNs require fixed-size inputs, which can be problematic for NLP tasks where input sequences vary in length.

➤ Limited Contextual Information:

- Many NLP tasks benefit from capturing long-range dependencies and understanding the broader context of a text, which is better addressed by models capable of modeling sequential data effectively.

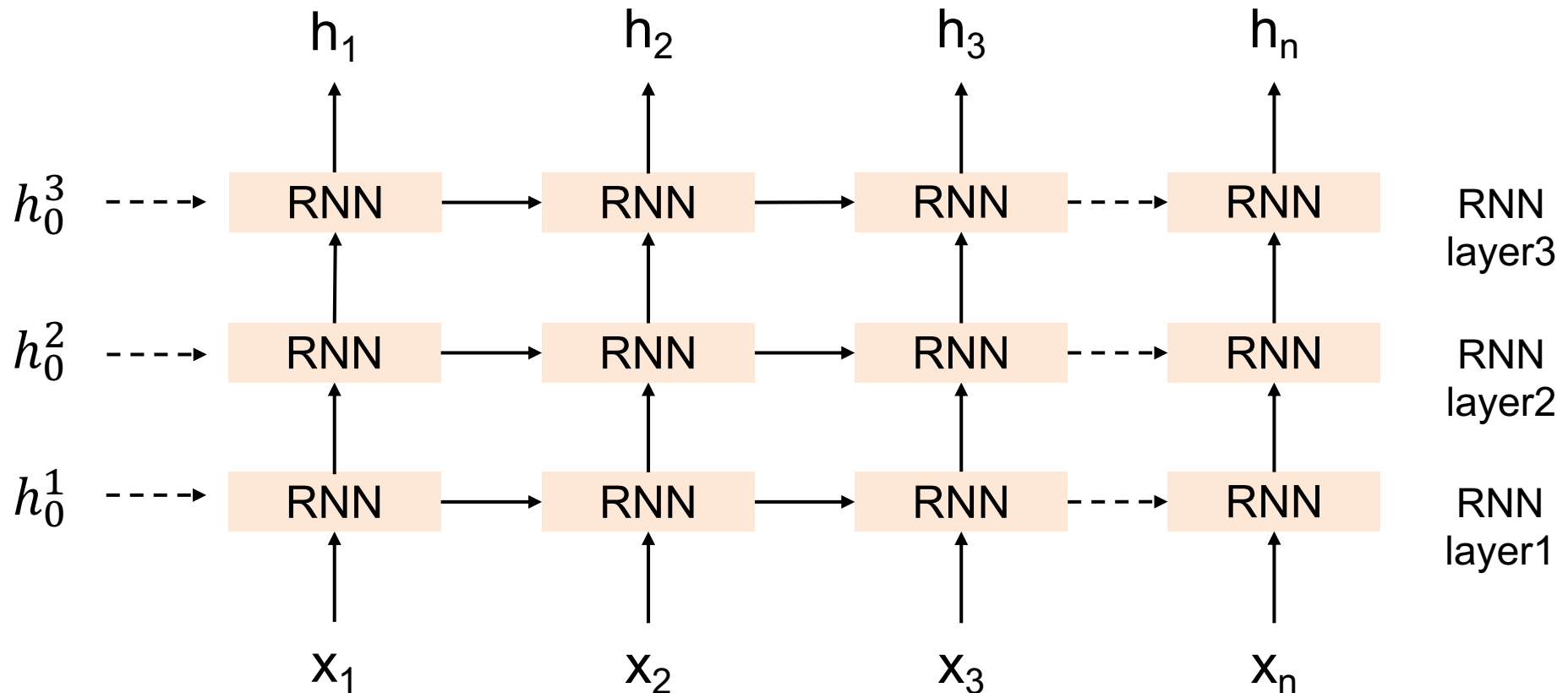


RNNs for Sequence Classification

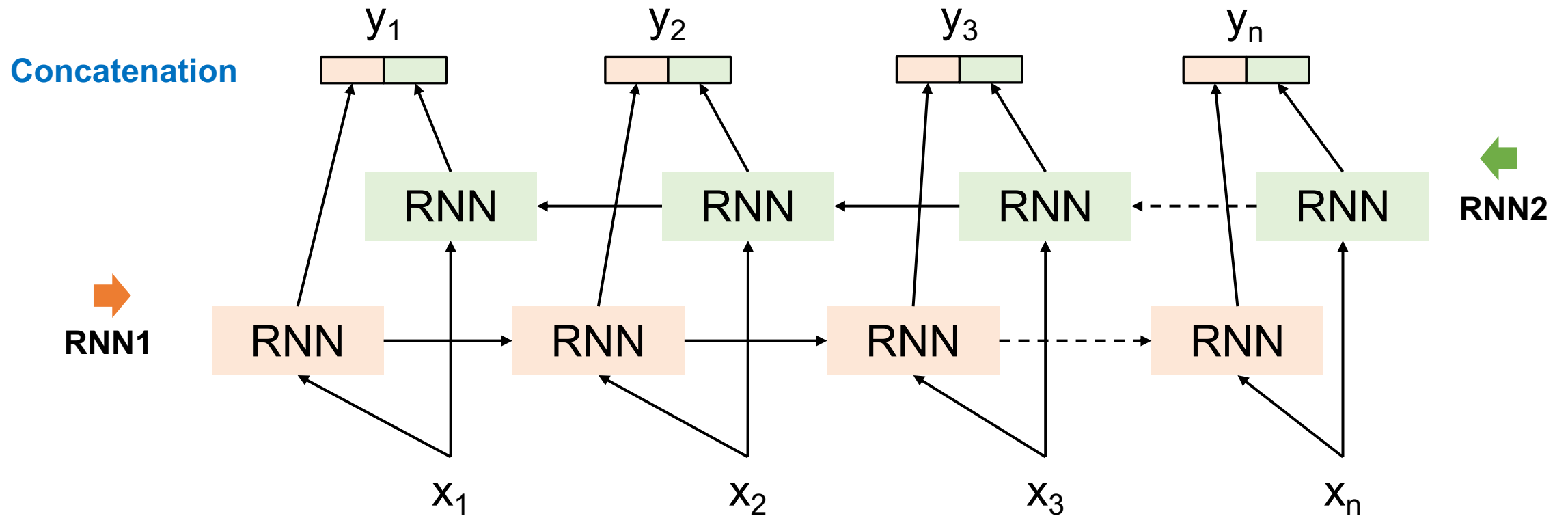


Stacked RNNs

- Stacked RNNs consist of multiple networks where the output of one layer serves as the input to a subsequent layer

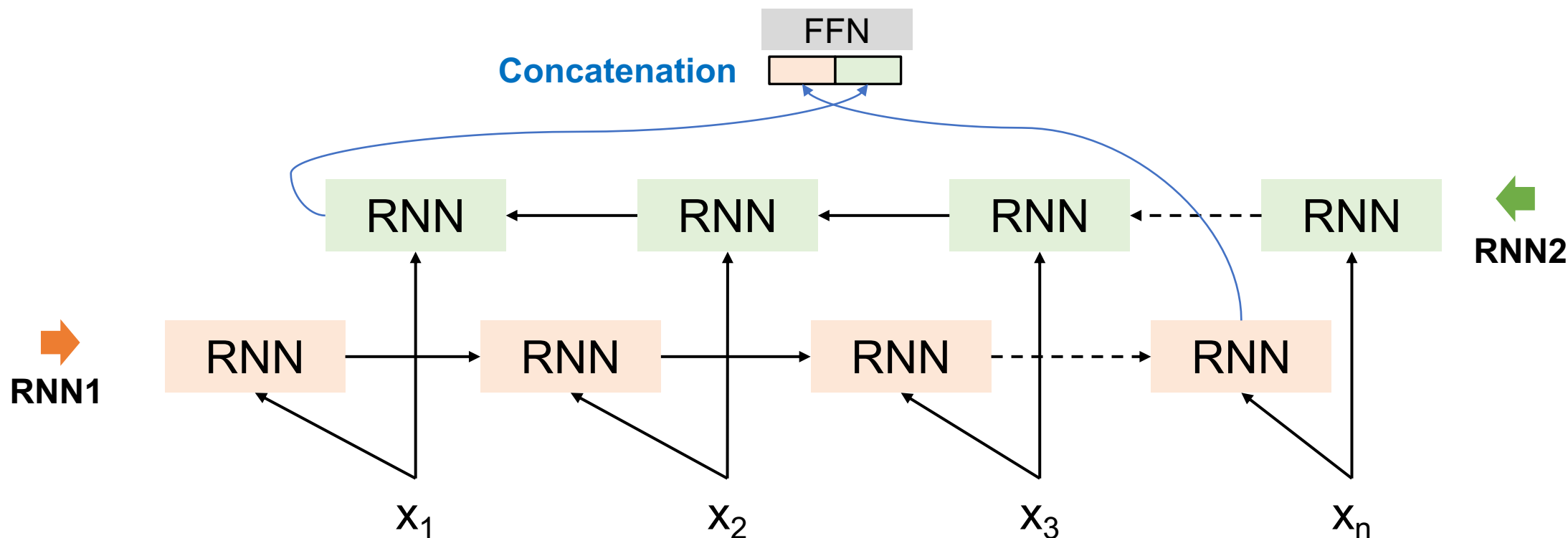


Bidirectional RNNs



Bidirectional RNNs for Sequence Classification

- The final hidden units from the forward and backward passes are combined to represent the entire sequence.
- This combined representation serves as input to the subsequent classifier.

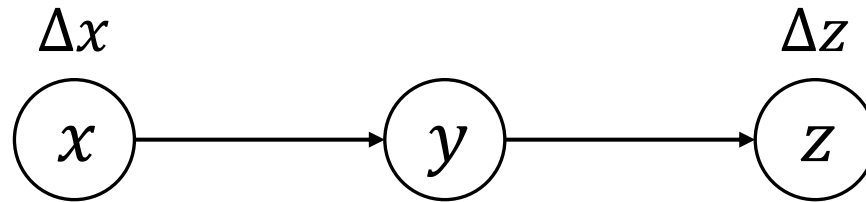


Shortage of standard RNNs

- Gradient vanishing (梯度消失) problem
- Long-term dependencies cannot be handled.
 - RNNs can only remember local relationships.
 - This is result from **back-propagation through-time (BPTT)** during training.



(Calculus) Chain Rule - 1

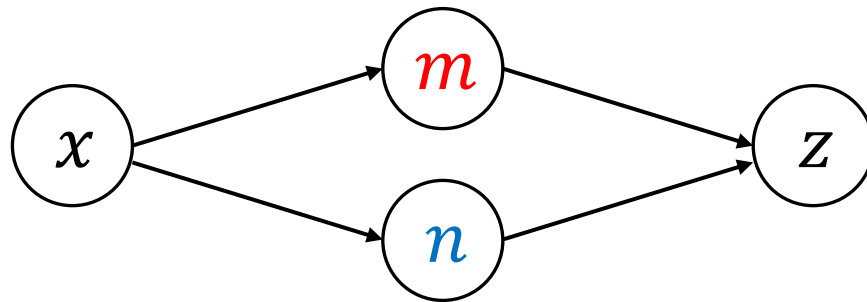


$$\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx}$$

- $\frac{dz}{dx}$ 代表 x 對 z 的影響，但 x 的變化會影響到 y ，接著 y 影響到 z



(Calculus) Chain Rule - 2



$$\frac{dz}{dx} = \frac{dz}{dm} \frac{dm}{dx} + \frac{dz}{dn} \frac{dn}{dx}$$

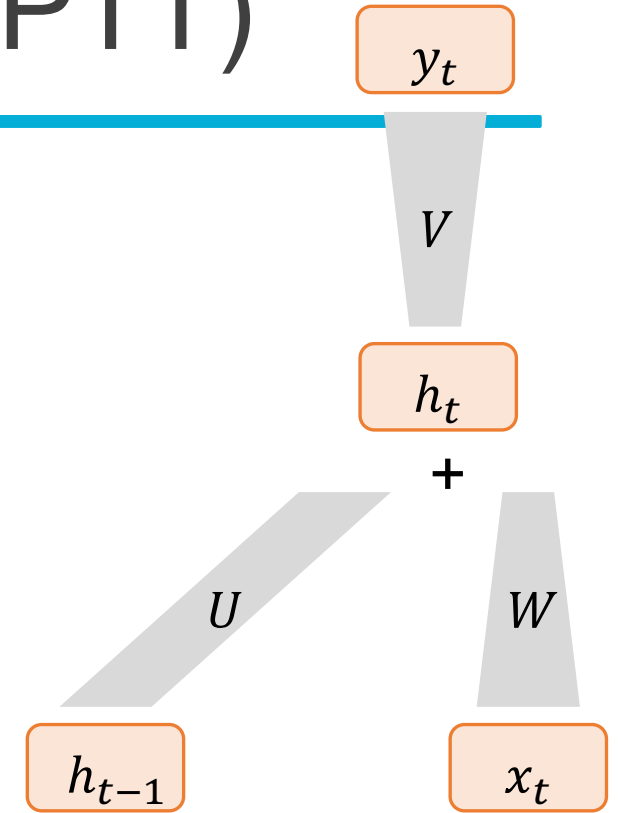
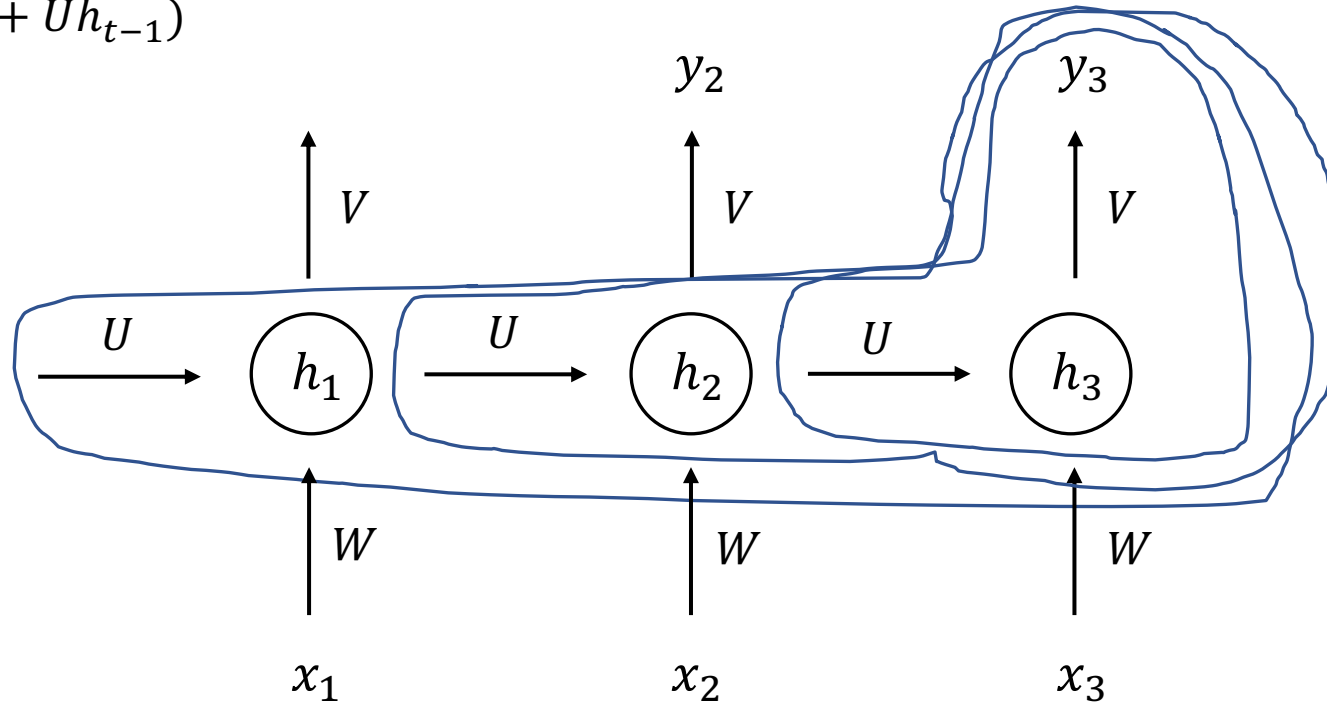
- $\frac{dz}{dx}$ 代表 x 對 z 的影響
 - 但 z 的變化由 m 和 n 共同影響



Back-propagation through-time (BPTT)

$$h_t = \tanh(Wx_t + Uh_{t-1})$$

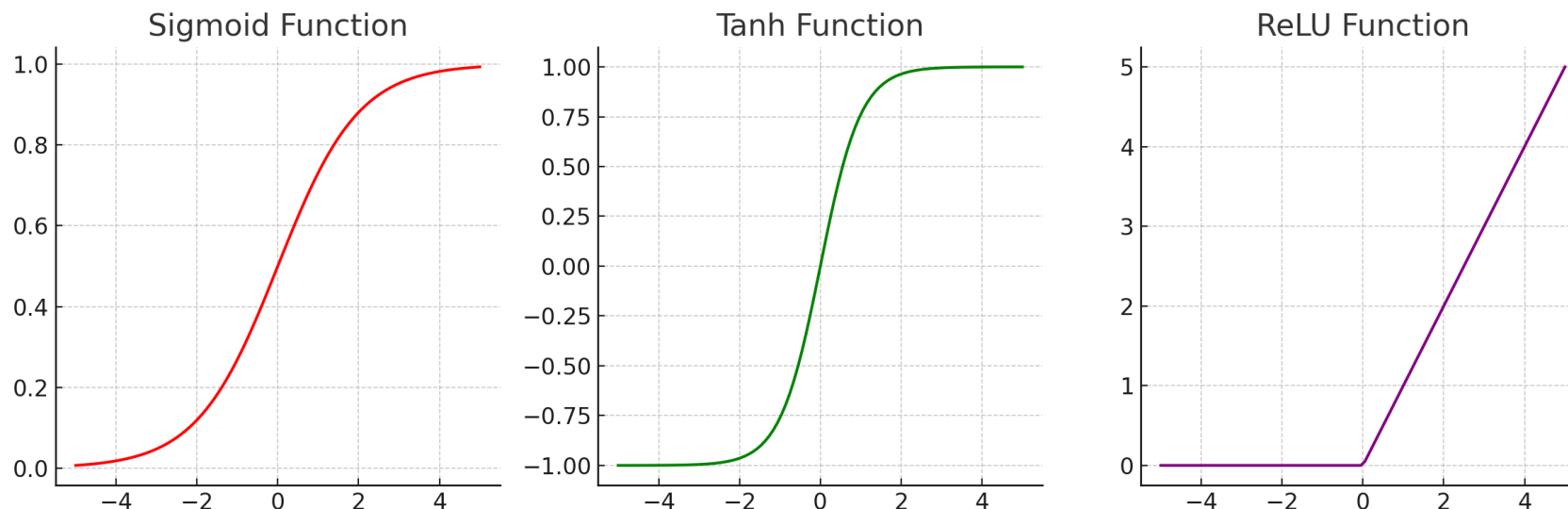
$$y_t = Vh_t$$



$$\frac{\partial \mathcal{L}}{\partial U} = \frac{\partial \mathcal{L}}{\partial y_3} \cdot \frac{\partial y_3}{\partial h_3} \cdot \frac{\partial h_3}{\partial U} + \frac{\partial \mathcal{L}}{\partial y_3} \cdot \frac{\partial y_3}{\partial h_3} \cdot \frac{\partial h_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial U} + \underbrace{\frac{\partial \mathcal{L}}{\partial y_3} \cdot \frac{\partial y_3}{\partial h_3} \cdot \frac{\partial h_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial U}}_{\text{(small)}}$$



Activation functions



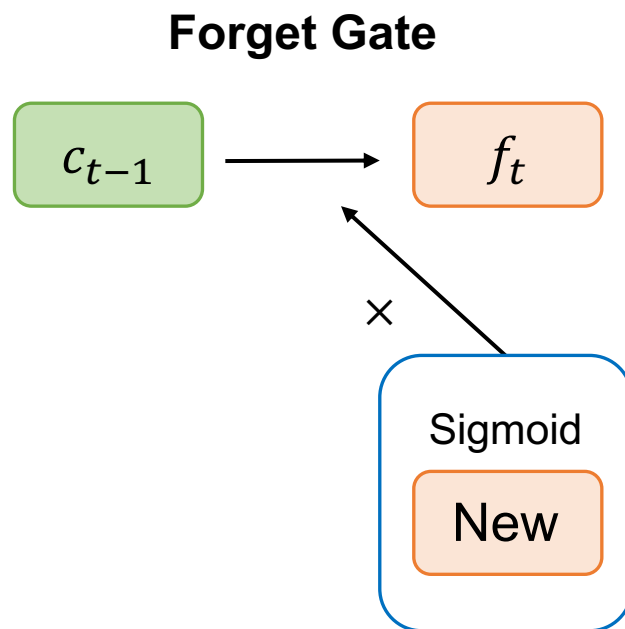
RNN 經常使用 tanh 的原因：

1. ReLU (unbounded) 可能會讓梯度變得非常大
2. Sigmoid 的數值是 0 到 1 之間，而 tanh 的數值在 -1 到 1 之間
3. tanh 在非飽和區比較陡峭



Gate (以 Forget Gate 為例)

更新記憶力的方式

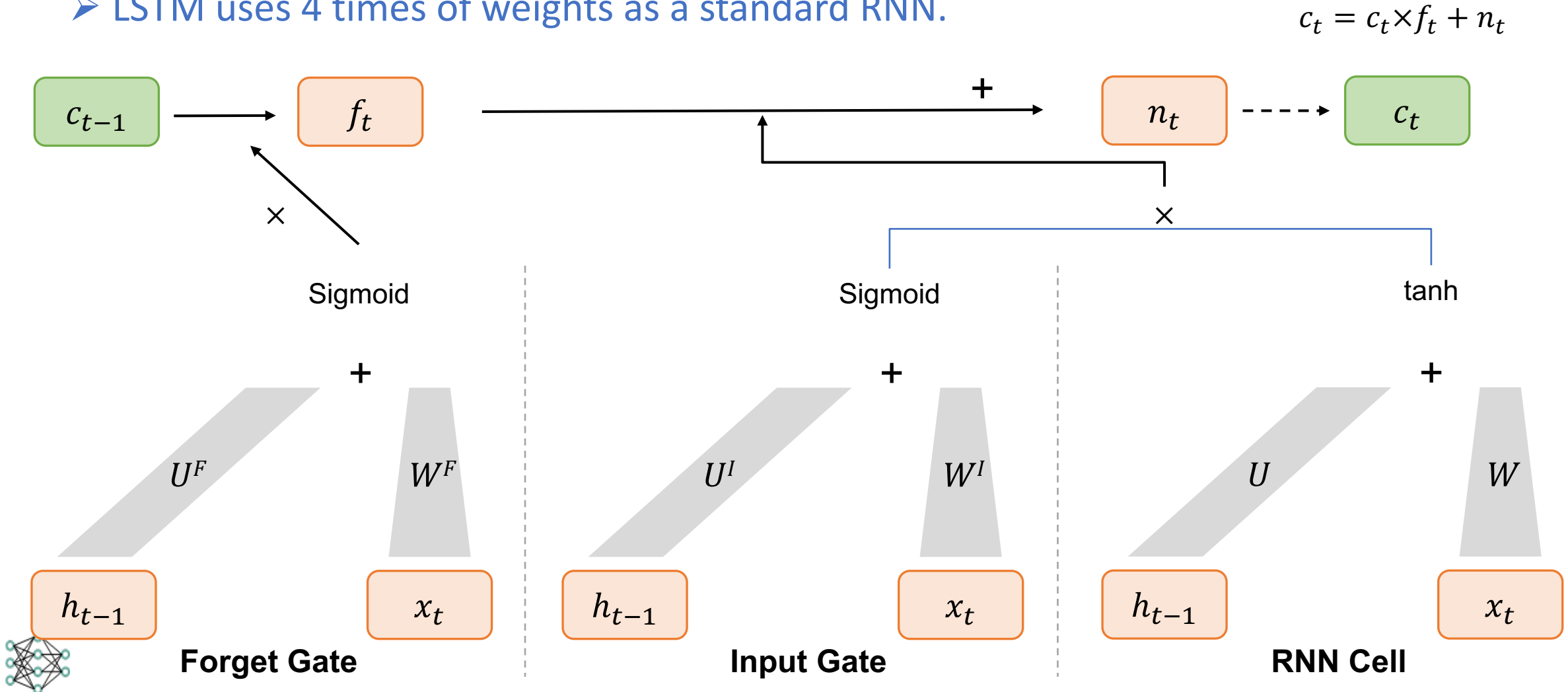


經 Sigmoid 之後產生一個比例



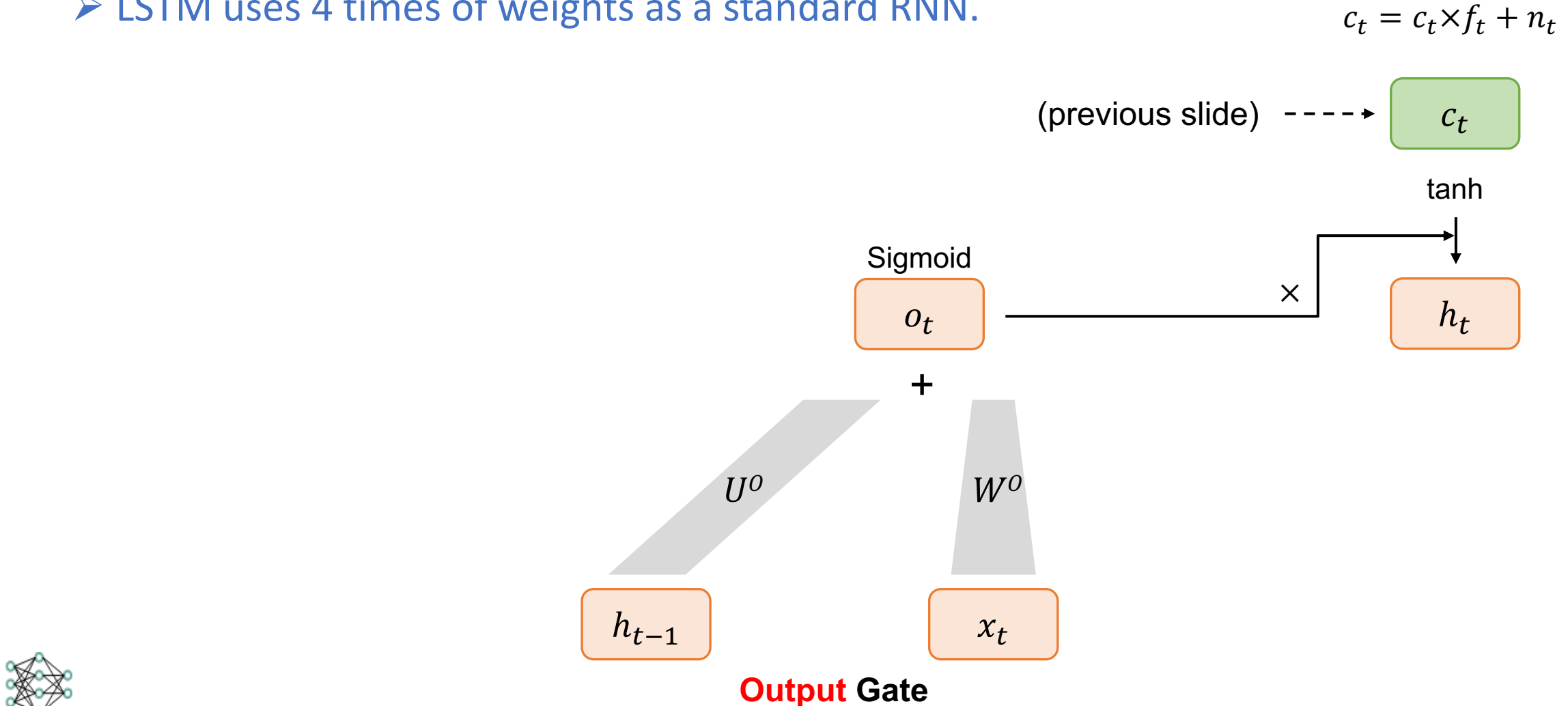
LSTM (Long Short-Term Memory)

➤ LSTM uses 4 times of weights as a standard RNN.



LSTM (Long Short-Term Memory)

- LSTM uses 4 times of weights as a standard RNN.



延伸學習

<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>



RNN vs. LSTM

	Memory Mechanism	Handles Long Dependencies?	Gradient Vanishing?
RNN	Hidden state h_t	Worse	Yes
LSTM	Memory state c_t + Hidden state h_t	Better	Much less than a standard RNN



Activation functions (啟動函數)

	值的範圍	使用情境
softmax	[0, 1] (sum up to 1)	需要讓多個值產生相加為一的機率值的時候，常用於建立分類任務模型時計算 cross-entropy loss
sigmoid	[0, 1]	需要讓單一值產生機率值的時候，常用於建立二元分類任務模型時計算 binary cross-entropy loss
tanh	[-1, 1]	常用於LSTM等RNN模型中，主要為引入非線性轉換。tanh的值是對稱的，以零為中心，在有些訓練條件下和sigmoid相比較不會梯度消失
ReLU	[0, 正值]	常用於CNN或MLP等模型中，主要為引入非線性轉換。計算簡單，在許多任務收斂快



Code Llama

<https://arxiv.org/abs/2308.12950>

Code Llama

 Meta AI

```
def create_evaluate_ops(task_prefix,
    data_format,
    input_paths,
    prediction_path,
    metric_fn_and_keys,
    validate_fn,
```



Figure source: <https://ai.meta.com/blog/code-llama-large-language-model-coding/>

Thank you!

Instructor: 林英嘉

 yjlin@cgu.edu.tw

TA: 劉美辰

 m1461014@cgu.edu.tw